Implementing Caching in EmployeeApp using Microsoft Garnet

Step-by-step implementation guide — Employee List Caching

Project: EmployeeApp.Api | **Target Framework:** .NET 10 | **Cache Server:** Microsoft Garnet (RESP/Redis-protocol compatible) | **Client Library:** Microsoft.Extensions.Caching.StackExchangeRedis

Contents

1. Overview

Microsoft Garnet is an open-source, high-performance remote cache-store created by Microsoft Research (`microsoft/garnet` on GitHub). It implements the RESP (REdis Serialization Protocol), which means any standard Redis client — including the official .NET client used by ASP.NET Core — can talk to it without any code changes. That is why this guide uses `Microsoft.Extensions.Caching.StackExchangeRedis` as the "caching tool": it is the standard `IDistributedCache` provider for ASP.NET Core, and it works against Garnet exactly as it would against Redis.

In this guide you will:

- Run a Garnet cache server locally as a .NET global tool (no Docker required)
- Add the `Microsoft.Extensions.Caching.StackExchangeRedis` package to `EmployeeApp.Api`

- Configure the connection using `appsettings.json` and a strongly typed options class (the same pattern already used for `RabbitMqOptions`)
- Register `IDistributedCache` in `Program.cs`
- Cache the employee list in `EmployeeService.GetAllAsync()` using the cache-aside pattern
- Invalidate the cached list whenever an employee is created, updated, or deleted
- Verify the cache is working

> **Note:** This guide only caches the *employee list* (`GetAllAsync`), as requested. `GetByIdAsync` is left untouched, but the same pattern can be applied to it later if needed.

2. Prerequisites

- .NET 10 SDK (already installed – matches the project's `net10.0` target framework)
- Visual Studio 2026 (already in use)
- Ability to install .NET global tools (`dotnet tool install --global`) to run Garnet locally — no Docker needed
- The existing `EmployeeApp.Api` project, with `EmployeeService`, `IEmployeeService`, `EmployeeController`, and the RabbitMQ options pattern already in place

3. Step 1 – Run the Garnet Cache Server

Garnet's default listening port is **6379** (the standard Redis port). Install and run it as a .NET global tool:

```
dotnet tool install --global Microsoft.Garnet
garnet-server --port 6379
```

Keep this running in its own PowerShell window while you develop (the same way you'd run a local RabbitMQ broker or SQL Server instance). This is what "Microsoft.Garnet" refers to in your request: it is the package/tool that provides the actual cache *server* process. Your ASP.NET Core app does not reference this package directly — it just connects to the running server over the network, the same way it already connects to RabbitMQ.

> **Running it long-term:** if you want Garnet available without keeping a terminal window open, you can register `garnet-server.exe` as a Windows Service (e.g., via `sc.exe create` or NSSM) instead of running it manually each time.

4. Step 2 – Add the Caching NuGet Package

From the solution root (`C:\reni\Test\EmployeeApp`), add the Redis-protocol distributed cache client to `EmployeeApp.Api`:

```
dotnet add EmployeeApp.Api\EmployeeApp.Api.csproj package
Microsoft.Extensions.Caching.StackExchangeRedis
```

This registers an `IDistributedCache` implementation backed by StackExchange.Redis, fully compatible with Garnet's RESP protocol.

5. Step 3 – Configure appsettings.json

Add a new `Garnet` section next to the existing `RabbitMq` section in `EmployeeApp.Api\appsettings.json`:

```
  "RabbitMq": {
    "HostName": "localhost",
    "Port": 5672,
    "UserName": "guest",
    "Password": "guest",
    "VirtualHost": "/",
    "Exchange": "employee.events",
    "EmployeeQueue": "employee.events.Queue"
  },
  "Garnet": {
    "ConnectionString": "localhost:6379",
    "InstanceName": "EmployeeApp:"
  }
```

6. Step 4 – Create GarnetOptions.cs

Add `EmployeeApp.Api\Options\GarnetOptions.cs`, mirroring the existing `RabbitMqOptions.cs` style:

```
namespace EmployeeApp.Api.Options
{
    public class GarnetOptions
    {
        public string ConnectionString { get; set; } = "localhost:6379";
        public string InstanceName { get; set; } = "EmployeeApp:";
    }
}
```

7. Step 5 – Register the Distributed Cache in Program.cs

Add this near the existing `builder.Services.Configure<RabbitMqOptions>(...)` line:

```
using EmployeeApp.Api.Options;

builder.Services.Configure<GarnetOptions>(builder.Configuration.GetSection("Garnet"));
builder.Services.AddStackExchangeRedisCache(options =>
{
    var garnetOptions = builder.Configuration.GetSection("Garnet").Get<GarnetOptions>() ??
new GarnetOptions();
    options.Configuration = garnetOptions.ConnectionString;
    options.InstanceName = garnetOptions.InstanceName;
});
```

8. Step 6 – Cache the Employee List (Cache-Aside Pattern)

Inject `IDistributedCache` into `EmployeeService` and update `GetAllAsync()` to check the cache first. On a miss, load from the repository, map to DTOs, then store the serialized result with an expiration:

```
public class EmployeeService(
    IEmployeeRepository repository,
    IMapper mapper,
    RabbitMqPublisher publisher,
    IDistributedCache cache) : IEmployeeService
{
    private const string EmployeeListCacheKey = "employees:all";

    private static readonly DistributedCacheEntryOptions CacheOptions = new()
    {
        AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5)
    };

    public async Task<List<EmployeeDto>> GetAllAsync()
    {
        var cached = await cache.GetStringAsync(EmployeeListCacheKey);
        if (!string.IsNullOrEmpty(cached))
        {
            return JsonSerializer.Deserialize<List<EmployeeDto>>(cached)!;
        }

        var employees = mapper.Map<List<EmployeeDto>>(await repository.GetAllAsync());

        await cache.SetStringAsync(
            EmployeeListCacheKey,
            JsonSerializer.Serialize(employees),
            CacheOptions);

        return employees;
    }
```

Add these `using` directives to `EmployeeService.cs`:

```
using Microsoft.Extensions.Caching.Distributed;
using System.Text.Json;
```

9. Step 7 – Invalidate the Cache on Create / Update / Delete

To keep the cached list from going stale, remove the cache entry whenever an employee is added, updated, or deleted, so the next `GetAllAsync()` call repopulates it from the database:

```
public async Task<EmployeeDto> AddAsync(EmployeeDto entity)
{
    var employee = mapper.Map<Employee>(entity);
    var savedEntity = await repository.CreateAsync(employee);
    var result = mapper.Map<EmployeeDto>(savedEntity);

    await cache.RemoveAsync(EmployeeListCacheKey);

    await publisher.PublishAsync(new Events.EmployeeEvent
    {
        EventType = "Created",
        EmployeeId = result.Id,
        EmployeeName = result.Name,
        OccurredAt = DateTime.UtcNow
    });

    return result;
}

public async Task<EmployeeDto> UpdateAsync(int id, EmployeeDto entity)
{
    var emp = mapper.Map<Employee>(entity);
    var updated = await repository.UpdateAsync(id, emp);

    await cache.RemoveAsync(EmployeeListCacheKey);

    return mapper.Map<EmployeeDto>(updated);
}

public async Task<EmployeeDto> DeleteAsync(int id)
{
    var deleted = await repository.DeleteAsync(id);
    var result = mapper.Map<EmployeeDto>(deleted);

    await cache.RemoveAsync(EmployeeListCacheKey);

    await publisher.PublishAsync(new Events.EmployeeEvent
    {
        EventType = "Deleted",
        EmployeeId = result.Id,
        EmployeeName = result.Name,
        OccurredAt = DateTime.UtcNow
    });

    return result;
}
```

10. Full Updated EmployeeService.cs

For reference, here is the complete file with caching applied:

```
using AutoMapper;
using EmployeeApp.Api.Messaging;
using EmployeeApp.Api.Models;
using EmployeeApp.Api.Models.Dtos;
using EmployeeApp.Api.Repositories;
using Microsoft.Extensions.Caching.Distributed;
using System.Text.Json;

namespace EmployeeApp.Api.Services
{
    public class EmployeeService(
        IEmployeeRepository repository,
        IMapper mapper,
        RabbitMqPublisher publisher,
        IDistributedCache cache) : IEmployeeService
    {
        private const string EmployeeListCacheKey = "employees:all";

        private static readonly DistributedCacheEntryOptions CacheOptions = new()
        {
            AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5)
        };

        public async Task<EmployeeDto> AddAsync(EmployeeDto entity)
        {
            var employee = mapper.Map<Employee>(entity);

            var savedEntity = await repository.CreateAsync(employee);
            var result = mapper.Map<EmployeeDto>(savedEntity);

            await cache.RemoveAsync(EmployeeListCacheKey);

            await publisher.PublishAsync(new Events.EmployeeEvent
            {
                EventType = "Created",
                EmployeeId = result.Id,
                EmployeeName = result.Name,
                OccurredAt = DateTime.UtcNow
            });

            return result;
        }

        public async Task<EmployeeDto> DeleteAsync(int id)
        {
            var deleted = await repository.DeleteAsync(id);
            var result = mapper.Map<EmployeeDto>(deleted);

            await cache.RemoveAsync(EmployeeListCacheKey);

            await publisher.PublishAsync(new Events.EmployeeEvent
```

```
            {
                EventType = "Deleted",
                EmployeeId = result.Id,
                EmployeeName = result.Name,
                OccurredAt = DateTime.UtcNow
            });

            return result;
        }

        public async Task<List<EmployeeDto>> GetAllAsync()
        {
            var cached = await cache.GetStringAsync(EmployeeListCacheKey);
            if (!string.IsNullOrEmpty(cached))
            {
                return JsonSerializer.Deserialize<List<EmployeeDto>>(cached)!;
            }

            var employees = mapper.Map<List<EmployeeDto>>(await repository.GetAllAsync());

            await cache.SetStringAsync(
                EmployeeListCacheKey,
                JsonSerializer.Serialize(employees),
                CacheOptions);

            return employees;
        }

        public async Task<EmployeeDto> GetByIdAsync(int id)
        {
            return mapper.Map<EmployeeDto>(await repository.GetByIdAsync(id));
        }

        public async Task<EmployeeDto> UpdateAsync(int id, EmployeeDto entity)
        {
            var emp = mapper.Map<Employee>(entity);
            var updated = await repository.UpdateAsync(id, emp);

            await cache.RemoveAsync(EmployeeListCacheKey);

            return mapper.Map<EmployeeDto>(updated);
        }
    }
}
```

11. Step 8 – Verify Caching Works

1. Start the Garnet server (Step 1) and run `EmployeeApp.Api`.

2. Call `GET /api/employees` twice (Swagger/OpenAPI UI, Postman, or the Blazor client). The first call is a cache miss (hits SQL Server); the second should be noticeably faster (served from Garnet).
3. Add temporary `ILogger` statements ("Cache hit" / "Cache miss") inside `GetAllAsync()` to confirm the behavior directly in the console/Serilog output – no extra tooling required.
4. (Optional) To inspect raw cache keys, connect a standalone RESP-compatible client such as **RedisInsight Desktop** (a Windows installer app, no Docker needed) to `localhost:6379` and run:

```
KEYS *
GET employees:all
```

5. Create, update, or delete an employee, then repeat step 4 – the `employees:all` key should be gone until the next `GetAll` call repopulates it.

12. Production Considerations

- **Connection resiliency:** configure retry/backoff (e.g., `ConfigurationOptions.AbortOnConnectFail = false`) so transient Garnet outages don't crash requests.
- **Key naming:** keep using an `InstanceName` prefix (e.g., `EmployeeApp:`) to avoid collisions if the same Garnet instance is shared across apps/environments.
- **Expiration strategy:** tune `AbsoluteExpirationRelativeToNow` (and/or add a sliding expiration) based on how often employee data changes.
- **Serialization:** `System.Text.Json` is fine to start; consider a binary format (e.g., MessagePack) later for larger payloads.
- **Security:** enable Garnet authentication (`--auth Password --password <secret>`) and store the secret via User Secrets/Key Vault, not in `appsettings.json`.
- **Cache stampede protection:** if load is high, guard the cache-miss path with a lock/semaphore per key to avoid many concurrent requests hitting SQL Server simultaneously.
- **Cache invalidation scope:** if you later cache `GetByIdAsync` too, make sure per-id keys are also invalidated on update/delete.
- **Monitoring:** track cache hit/miss ratio and Garnet server health alongside your existing Serilog sinks.

13. Quick Reference – Commands

Action	Command
Install the Garnet global tool	`dotnet tool install --global Microsoft.Garnet`
Start the Garnet server	`garnet-server --port 6379`
Add caching package	`dotnet add EmployeeApp.Api\EmployeeApp.Api.csproj package Microsoft.Extensions.Caching.StackExchangeRedis`
List all cache keys (via a RESP client, e.g. RedisInsight)	`KEYS *`
Inspect the cached employee list	`GET employees:all`
Clear the whole cache	`FLUSHDB`
Stop the Garnet server	Press `Ctrl+C` in its console window

14. Summary

By running Microsoft Garnet locally as a .NET global tool and wiring it into ASP.NET Core's standard `IDistributedCache` abstraction via `Microsoft.Extensions.Caching.StackExchangeRedis`, the employee list served by `EmployeeService.GetAllAsync()` is cached for a configurable window and automatically invalidated on create, update, and delete — reducing load on SQL Server while keeping the data consistent, following the same configuration-driven pattern already established for RabbitMQ in this project.